

Objects – Simple Hinglish Summary

Source: <https://javascript.info/object>

Main baat

JavaScript me **8 data types** hain. Inme se 7 **primitive** hain kyunki unki value me sirf **ek cheez** hoti hai (string, number, etc.).

Objects alag hain: wo **keyed collections** (key ke saath data) aur complex cheezein store karte hain. JavaScript me objects **lagbhag har jagah** hote hain, isliye inhe pehle samajhna zaruri hai.

Object kaise banate hain: curly braces `{...}` me optional **properties** ki list ke saath. Property ek **"key: value"** pair hai. `key` string hoti hai (property name), `value` kuch bhi ho sakti hai.

Analogy: Object ko ek **almari** samjho jisme label lagi files hain. Har data apni key (label) ke saath store hota hai. Naam se file dhundhna, ya file add/remove karna aasan hai.

Khali object banane ke do tarike:

```
javascript

let user = new Object(); // "object constructor" syntax
let user = {};           // "object literal" syntax
```

Aam taur par `{...}` use hota hai. Ise **object literal** kehte hain.

1. Literals aur properties

```
javascript

let user = {           // ek object
  name: "John",        // key "name" par value "John"
  age: 30               // key "age" par value 30
};
```

- Colon `:` se pehle **key** (name/identifier) aur baad me **value**.
- Dot notation se value milti hai:

```
javascript

alert( user.name ); // John
alert( user.age );  // 30
```

- Value kisi bhi type ki ho sakti hai. Boolean add karo:

```
javascript  
  
user.isAdmin = true;
```

- Property hatane ke liye `delete` operator:

```
javascript  
  
delete user.age;
```

- **Multiword** property name ho to use **quotes** me likhna padta hai:

```
javascript  
  
let user = {  
  name: "John",  
  age: 30,  
  "likes birds": true // multiword name quoted  
};
```

- Aakhri property ke baad **comma** laga sakte ho (trailing comma). Isse properties add/remove/move karna aasan hota hai kyunki saari lines ek jaisi hoti hain:

```
javascript  
  
let user = {  
  name: "John",  
  age: 30,  
}
```

2. Square brackets

Multiword property par **dot access** kaam nahi karta:

```
javascript  
  
user.likes birds = true // syntax error
```

Dot ke liye key ek **valid variable identifier** honi chahiye: space nahi, digit se shuru nahi, special characters nahi (`$`) aur `_` allowed).

Square bracket notation kisi bhi string ke liye kaam karti hai:

javascript

```
let user = {};  
  
// set  
user["likes birds"] = true;  
  
// get  
alert(user["likes birds"]); // true  
  
// delete  
delete user["likes birds"];
```

Brackets ke andar string **quotes** me hoti hai (koi bhi quote chalega).

Bada fayda: brackets me **variable** ya **koi bhi expression** de sakte ho:

javascript

```
let key = "likes birds";  
  
user[key] = true; // user["likes birds"] = true ke barabar
```

key runtime par calculate ho sakti hai ya user input par depend kar sakti hai.

javascript

```
let user = {  
  name: "John",  
  age: 30  
};  
  
let key = prompt("What do you want to know about the user?", "name");  
  
alert( user[key] ); // John (agar "name" daala)
```

Dot notation aise nahi chalti:

javascript

```
let key = "name";  
alert( user.key ) // undefined (key naam ki property dhundhta hai)
```

Computed properties

Object literal banate waqt bhi square brackets use kar sakte ho:

```
javascript

let fruit = prompt("Which fruit to buy?", "apple");

let bag = {
  [fruit]: 5, // property ka naam fruit variable se aata hai
};

alert( bag.apple ); // 5 (agar fruit = "apple")
```

Ye `bag[fruit] = 5;` ke barabar hai, bas dikhne me achha hai. Complex expressions bhi chalte hain:

```
javascript

let fruit = 'apple';
let bag = {
  [fruit + 'Computers']: 5 // bag.appleComputers = 5
};
```

Kab kya use karein: Naam pata aur simple ho to **dot** use karo. Kuch complex chahiye to **square brackets**.

3. Property value shorthand

Real code me aksar existing variables ko property ki value banate hain:

```
javascript

function makeUser(name, age) {
  return {
    name: name,
    age: age,
    // ...
  };
}
```

Property aur variable ka naam same ho to **shorthand**:

```
javascript
```

```
function makeUser(name, age) {  
  return {  
    name, // name: name ke barabar  
    age,  // age: age ke barabar  
    // ...  
  };  
}
```

Normal properties aur shorthand ek hi object me mix kar sakte ho:

```
javascript  
  
let user = {  
  name, // name:name ke barabar  
  age: 30  
};
```

4. Property names ki limitations

Variable ka naam reserved words (`for`, `let`, `return`) nahi ho sakta. Lekin **object property** ke liye ye restriction nahi hai:

```
javascript  
  
let obj = {  
  for: 1,  
  let: 2,  
  return: 3  
};  
  
alert( obj.for + obj.let + obj.return ); // 6
```

Property names koi bhi **strings** ya **symbols** ho sakte hain. Dusre types automatically **string** me convert ho jate hain:

```
javascript  
  
let obj = {  
  0: "test" // "0": "test" ke barabar  
};  
  
alert( obj["0"] ); // test  
alert( obj[0] );  // test (same property)
```

Ek chhota gotcha: special property `__proto__` ko non-object value par set nahi kar sakte:

```
javascript

let obj = {};
obj.__proto__ = 5;
alert(obj.__proto__); // [object Object] - 5 ignore ho gaya
```

(Iski detail aage prototype ke chapters me hai.)

5. Property exist karti hai ya nahi: `in` operator

JS me objects ki khaas baat: **kisi bhi property ko access kar sakte ho**. Property na ho to **error nahi aata**, bas `undefined` milta hai.

```
javascript

let user = {};

alert( user.noSuchProperty === undefined ); // true ka matlab "aisi property na"
```

Iske liye `in` operator bhi hai:

```
javascript

"key" in object
```

```
javascript

let user = { name: "John", age: 30 };

alert( "age" in user );    // true
alert( "blabla" in user ); // false
```

`in` ke left side par **property ka naam** hona chahiye (aam taur par quoted string). Quotes hata do to iska matlab hai ki variable me asli naam hai:

```
javascript

let user = { age: 30 };

let key = "age";
alert( key in user ); // true
```

`in` operator kyu hai? `undefined` se compare kyu nahi? Aam taur par `undefined` se compare karna chalta hai. Lekin ek special case me wo fail hota hai: jab property exist karti hai lekin uski value `undefined` hai:

javascript

```
let obj = {
  test: undefined
};

alert( obj.test );      // undefined (to kya property nahi hai?)
alert( "test" in obj ); // true (property hai!)
```

Aisi situations bahut rare hain, kyunki `undefined` explicitly assign nahi karna chahiye. "Unknown/empty" ke liye aam taur par `null` use hota hai.

6. `for...in` loop

Object ki saari keys par chalne ke liye special loop `for...in` hai. Ye `for(;;)` se **bilkul alag** hai.

javascript

```
for (key in object) {
  // har key ke liye body chalti hai
}
```

javascript

```
let user = {
  name: "John",
  age: 30,
  isAdmin: true
};

for (let key in user) {
  alert( key );      // name, age, isAdmin
  alert( user[key] ); // John, 30, true
}
```

- Loop variable loop ke andar declare kar sakte ho (`let key`).
- `key` ki jagah koi aur naam (jaise `prop`) bhi chalta hai.

Order kaise hota hai?

Kya objects ordered hote hain? **Ek special tarike se: integer properties sort hoti hain, baaki creation order me aati hain.**

Example (phone codes):

```
javascript

let codes = {
  "49": "Germany",
  "41": "Switzerland",
  "44": "Great Britain",
  "1": "USA"
};

for (let code in codes) {
  alert(code); // 1, 41, 44, 49
}
```

Hum chahte the ki (49) pehle aaye, lekin codes **ascending sorted order** me aate hain ((1,
41, 44, 49)) kyunki wo integers hain.

"Integer property" kya hai? Wo string jo integer me convert karke wapas string banane par **same rehti hai**. "49" integer property hai, lekin " +49" aur "1.2" nahi hain.

Non-integer keys creation order me aati hain:

```
javascript

let user = {
  name: "John",
  surname: "Smith"
};
user.age = 25;

for (let prop in user) {
  alert(prop); // name, surname, age
}
```

Phone codes ki problem ka fix: codes ko non-integer bana do, har code se pehle " +" laga do:

```
javascript
```

```
let codes = {
  "+49": "Germany",
  "+41": "Switzerland",
  "+44": "Great Britain",
  "+1": "USA"
};

for (let code in codes) {
  alert( +code ); // 49, 41, 44, 1
}
```

Summary

Objects **associative arrays** hain jinme kuch special features hain. Wo properties (key-value pairs) store karte hain:

- Keys **strings** ya **symbols** honi chahiye (aam taur par strings).
- Values **kisi bhi type** ki ho sakti hain.

Property access karne ke tarike:

- Dot notation: `obj.property`
- Square brackets: `obj["property"]`. Brackets se key variable se le sakte ho:
`obj[varWithKey]`

Extra operators:

- Property delete karna: `delete obj.prop`
- Property exist karti hai ya nahi: `"key" in obj`
- Object par iterate karna: `for (let key in obj)`

Jo hum ne padha use "**plain object**" ya sirf `Object` kehte hain.

JavaScript me aur bhi objects hain:

- `Array`: ordered data collections ke liye
- `Date`: date aur time ke liye
- `Error`: error ki jaankari ke liye

Log kabhi kabhi "Array type" ya "Date type" kehte hain, lekin ye asal me alag types nahi, sab `object` type ke hi hain aur use alag tarike se extend karte hain.

Practice Tasks (Answers)

1. Hello, object:

```
javascript
```

```
let user = {};  
user.name = "John";  
user.surname = "Smith";  
user.name = "Pete";  
delete user.name;
```

2. `isEmpty(obj)` (object khali hai ya nahi):

javascript

```
function isEmpty(obj) {  
  for (let key in obj) {  
    // loop shuru hua matlab property hai  
    return false;  
  }  
  return true;  
}
```

3. Salaries ka sum:

javascript

```
let salaries = {  
  John: 100,  
  Ann: 160,  
  Pete: 130  
};  
  
let sum = 0;  
for (let key in salaries) {  
  sum += salaries[key];  
}  
  
alert(sum); // 390
```

4. Numeric values ko 2 se multiply karo:

javascript

```
function multiplyNumeric(obj) {
  for (let key in obj) {
    if (typeof obj[key] == 'number') {
      obj[key] *= 2;
    }
  }
}
```

Quick Summary

Kaam	Syntax
Object banana	<code>let user = { name: "John", age: 30 };</code>
Value padhna	<code>user.name</code> ya <code>user["name"]</code>
Naya add / badalna	<code>user.isAdmin = true;</code>
Hatana	<code>delete user.age;</code>
Multiword key	<code>"likes birds": true</code> aur <code>user["likes birds"]</code>
Variable se key	<code>user[key]</code> (dot yahan nahi chalta)
Computed property	<code>{ [fruit]: 5 }</code>
Shorthand	<code>{ name, age }</code>
Property hai ya nahi	<code>"age" in user</code>
Saari keys par loop	<code>for (let key in user)</code>
Order	Integer keys sorted, baaki creation order me